

❖ B ve B+ Ağaçlarının Günümüzde Kullanım Alanları ve Mimari Analizi

Veri yapıları üzerine yaptığım araştırmalarda, özellikle büyük veri yığınlarını yönetmek için en sık karşıma çıkan yapılar **B-Ağacı (B-Tree)** ve onun daha gelişmiş bir versiyonu olan **B+ Ağacı (B+ Tree)** oldu. Temel olarak bu yapılar, verileri bellekte rastgele tutmak yerine, hiyerarşik ve sıralı bir şekilde organize ederek arama sürelerini kısaltmayı amaçlıyor. Özellikle verinin diske yazıldığı ve diskten okunduğu durumlarda (Disk I/O işlemlerinde) klasik ikili arama ağaçlarının (Binary Search Tree) yetersiz kaldığını ve bu noktada **B/B+ ağaçlarının** devreye girdiğini gördüm.

❖ B-Tree ve B+ Tree Nedir, Temel Farkları ve Özellikleri Nelerdir?

Araştırmalarım sonucunda bu iki yapının temelde arama sürelerini optimize etmek için tasarlandığını, ancak mimari olarak önemli bir farka sahip olduklarını gördüm :

- **B-Tree (B-Ağacı):** Kendi kendini dengeleyen çok yollu bir arama ağacıdır. En büyük özelliği, hem yönlendirici anahtarların hem de gerçek verilerin (data) ağacın tüm düğümlerinde (iç düğümler dahil) saklanabilmesidir.
 - **B+ Tree (B+ Ağacı):** B-Tree yapısının modern sistemler için güncellenmiş halidir. Burada gerçek veriler **sadece en alttaki yaprak düğümlerde (leaf nodes)** saklanır. İç düğümler ise sadece yönlendirme amaçlı indeksleri tutar. Ayrıca tüm yaprak düğümler birbirine bağlı bir liste (linked list) şeklinde entegre edilmiştir, bu da sıralı okumaları inanılmaz derecede hızlandırır.
-

B ve B+ Ağaçları Veri Tabanlarında (MySQL, PostgreSQL, MongoDB) Nasıl ve Hangi Amaçla Kullanılır?

Veri tabanlarında milyonlarca satır arasından bir kaydı bulmak için "indeksleme" mekanizmaları kullanılır ve bu mekanizmaların arkasında bu ağaç yapıları yatar. İncelediğim popüler sistemlerdeki durum şu şekildedir:

- **MySQL (InnoDB):** MySQL'in resmi dokümantasyonunu incelediğimde, varsayılan depolama motoru olan InnoDB'nin tamamen **B+ Tree** yapısını kullandığını gördüm. Tablolardaki "Primary Key" (Birincil Anahtar) indeksleri bu ağaç mimarisıyla tutulur. Sistem milyarlarca veri arasından aranan ID'yi kökten yaprağa doğru birkaç adımda bulur.
- **PostgreSQL:** Bu veri tabanı sistemi ise varsayılan indeksleme algoritması olarak **B-Tree** altyapısını tercih eder. Özellikle "büyüktür, küçüktür, eşittir" (>, <, =) gibi

operatörleri içeren aralık ve karşılaştırma sorgularını hızlandırmada çok başarılıdır.

- **MongoDB:** NoSQL mimarisine sahip olan MongoDB'de dökümanlar üzerinde bir indeks (örneğin kullanıcı adına göre) oluşturduğumuzda, sorguların tüm koleksiyonu taramasını engellemek için arka planda yine B-Tree mantığına dayalı indeks blokları inşa edilir.

İşletim Sistemlerinde ve Dosya Sistemlerinde Bu Yapılardan Nasıl Yararlanılır?

Bilgisayarımızdaki işletim sistemleri disk üzerindeki klasör hiyerarşisini ve dosya konumlarını yönetmek için bu yapılardan yararlanır. Windows'un kullandığı **NTFS**, Linux'un kullandığı **Ext4** veya **Btrfs** gibi modern dosya sistemleri, dosya isimlerini ve disk üzerindeki fiziksel blok adreslerini B+ Tree yapılarıyla haritalandırır. Bir klasörün içinde on binlerce dosya olsa bile, aradığımız dosyaya çift tıkladığımızda anında açılabilmesinin sebebi işletim sisteminin diski baştan sona taramak yerine bu ağaç yapısı üzerinden doğrudan dosyanın adresine zıplamasıdır.

Arama Motorları ve Büyük Veri Sistemlerinde B ve B+ Ağaçlarının Rolü Nedir?

Google gibi arama motorları milyarlarca web sayfasını tarayıp kelimeleri haritalandırırken "ters indeks" (inverted index) yapısını kullanır. Kullanıcı bir kelime arattığında milisaniyeler içinde sonuç dönebilmek için bu devasa kelime havuzu B-Tree türevi yapılarla indekslenir. Büyük veri sistemlerinde ise ardışık log kayıtlarının analizinde B+ Tree'nin yaprak düğümlerinin birbirine bağlı olması büyük avantaj sağlar; belirli bir zaman aralığına ait veriler disk üzerinde ardışık olarak çok hızlı bir şekilde taranabilir.

Soru 5: Bu Yapılar Modern Sistemlerde Neden Özellikle Tercih Edilmektedir?

Araştırmalarım doğrultusunda modern sistemlerin bu yapıları seçmesinin en kritik nedenlerini şu şekilde sıralayabilirim:

1. **Disk Erişimini (I/O) Minimize Etmesi:** Diskten okuma yapmak RAM'e göre çok yavaştır. Bu ağaçlar yatayda geniş ve sık tasarlandığı için minimum disk okumasıyla hedefe ulaşır.
2. **Dengeli Yapı (Self-Balancing):** Ağaca ne kadar veri eklenirse eklensin yapısı bozulmaz ve dengede kalır. Bu da her arama işleminin her zaman aynı yüksek performansla çalışmasını sağlar.

3. **Sıralı Erişim Kolaylığı:** Özellikle B+ Tree'de yapraklar birbirine bağlı olduğu için, verileri baştan sona sıralı okumak (örneğin "A'dan Z'ye sırala" demek) ek bir sıralama maliyeti gerektirmez.

B ve B+ Ağaçlarının Çalışma Mantığını Günlük Hayattan Hangi Örneklerle Somutlaştırabiliriz?

Bu soyut algoritmaları günlük hayatta asırlardır kullandığımız fiziksel arşivleme mantığına benzetebiliriz:

- **Hastane Arşiv Sistemi (B+ Tree Benzetmesi):** Milyonlarca hasta dosyasının olduğu fiziksel bir arşivi düşünelim. Görevliden "2023 yılı Mart ayına ait kayıtları" istediğimizde tüm arşivi gezmez. Girişteki tabelalara bakar (Kök düğüm), 2023 koridoruna yönelir, Mart çekmecesini açar (İç düğümler). Çekmeceyi açtığında tüm dosyalar 1 Mart'tan 31 Mart'ye kadar yan yana dizilmiştir (Bağlantılı yaprak düğümler). Görevli o aralığı tek seferde çeker alır.
- **Telefon Rehberi ve Kütüphane (B-Tree Benzetmesi):** Bir telefon rehberinde isim ararken ilk sayfadan başlamayız; kenardaki harf fihristlerinden doğrudan ilgili harfe atlarız. Kütüphanede de kitap ararken önce katı, sonra kategoriye (örneğin Tarih), sonra yazar soyadını bularak hedefe ulaşırız. Bu hiyerarşik filtreleme adımları tamamen bu ağaçların çalışma mantığıdır.

Bu Veri Yapıları Neden Önemlidir ve Büyük Veri Çağında Neden İhtiyaç Duyulmaktadır?

Bence bu veri yapılarının en büyük önemi, bilgisayar donanımındaki en büyük darboğaz olan "yavaş disk hızı" sorununu yazılımsal bir zekayla çözmüş olmalarıdır. Donanımın fiziksel sınırlarını optimize ederler.

Büyük veri çağında ise saniyede petabaytlarca veri üretiliyor. Eskiden az veri varken doğrusal arama (baştan sona okuma) bir şekilde iş görüyordu. Ancak günümüzün sosyal medya, e-ticaret ve bankacılık sistemlerinde veriyi eklerken anında düzene sokan bu yapılar olmasaydı, sunucuların arama sorgularına yanıt vermesi dakikalar hatta saatler sürerdi. Bu yüzden büyük veri çağının gizli kahramanlarıdır diyebilirim.

Sizce Bu Veri Yapıları Gelecekte de Kullanılmaya Devam Edecek midir?

Bence kesinlikle kullanılmaya devam edecek. Evet, bugün SSD ve NVMe teknolojileri sayesinde diskler çok hızlandı, hatta veri yazmayı kolaylaştıran LSM-Tree gibi yeni yapılar da türedi. Ama sonuçta bilgisayarlar veriyi arka planda hâlâ "bloklar" halinde saklıyor. Bu mantık kalıcı olduğu sürece, veriyi düzenlemenin ve aramanın en sağlam, en güvenilir yolu yine B ve B+ ağaçları olacaktır. Yani bu yapılar geçici bir heves değil, bilgisayar dünyasının asla eskimeyecek temel taşıdır.

Mehmet Şükrü Aybar / 02240224162

Yazılım Mühendisliği